

R_sine

Drawing a Sine Wave with a myCobot 280 — Detailed Explanation

Modified-DH forward kinematics · velocity-propagation Jacobian · damped-least-squares
inverse kinematics · ROS 2 (Jazzy) · independent verification against ikpy

Verification at a glance

Check	Result
Forward kinematics (DH) vs URDF	8.9e-16 m
Velocity-propagation Jacobian vs finite diff.	2.5e-07
Analytical IK tracking of the sine	0.001 mm
Mean IK iterations per waypoint	16
Max 3D reach / horizontal reach	0.426 m / 0.301 m
Home tool position (all joints 0)	(0.061, 0.046, 0.410) m

All numbers are reproduced by the scripts in **analysis/**.

R_sine — Drawing a Sine Wave with a myCobot 280

Analytical kinematics + inverse kinematics on a 6-DOF arm (ROS 2)

This project drives a myCobot 280 — a small six-degree-of-freedom robotic arm — so that its end-effector traces a sine wave on a flat surface such as a vertical board. Rather than relying only on a black-box library, the kinematics are derived from first principles using the Modified (Craig) Denavit-Hartenberg convention, a velocity-propagation Jacobian, and a damped-least-squares inverse kinematics solver. Every stage is verified numerically, and an independent solver (ikpy) is used as a cross-check. The system runs as clean, separate ROS 2 nodes and is visualized in RViz.

- Goal: make a 6-DOF myCobot 280 trace a sine wave on a surface
- Built from first principles: Modified-DH kinematics, velocity-propagation Jacobian, damped-least-squares IK
- Every result cross-checked against an independent library (ikpy)
- Runs in ROS 2 (Jazzy); visualized in RViz

myCobot 280 — home configuration ($q = 0$)
EE at (0.061, 0.046, 0.410) m

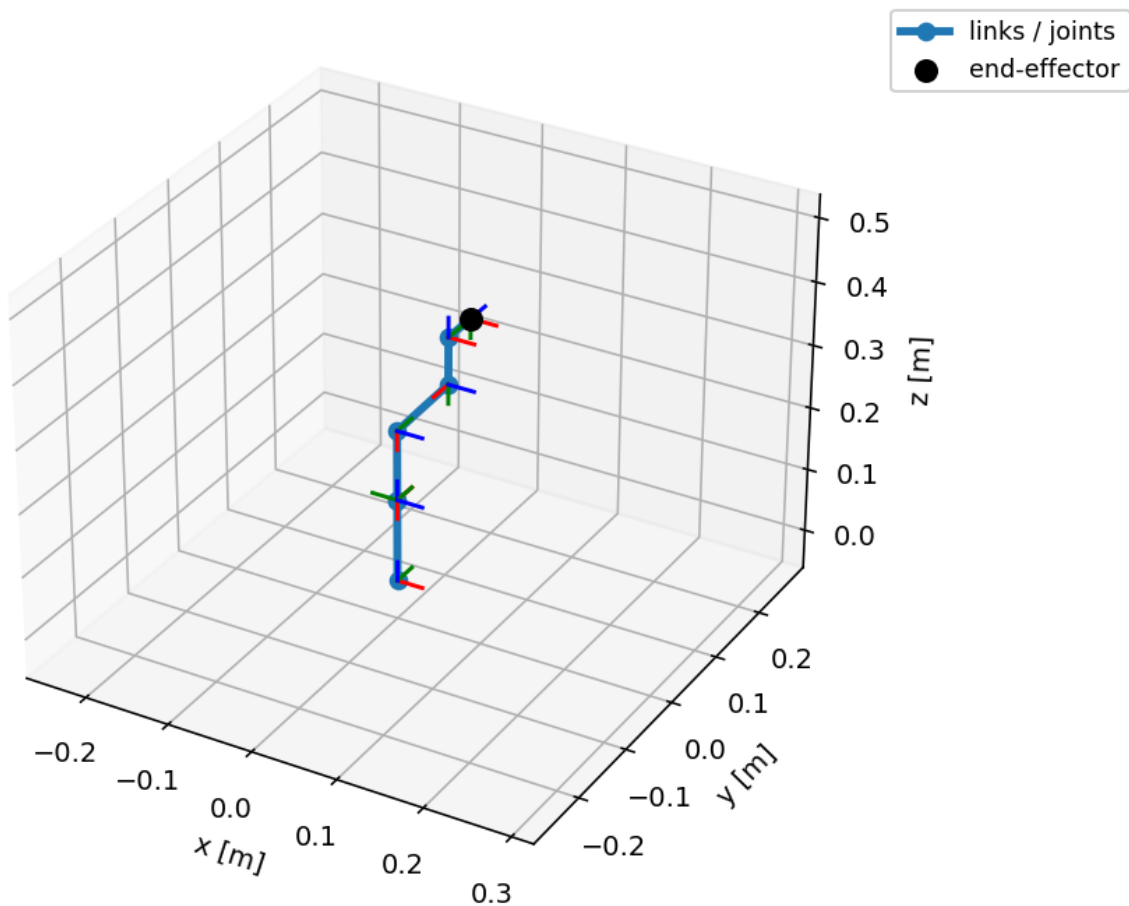


Figure: fig_home_pose.png

Objective and Method

The work is organized around the three classical kinematics problems. Forward kinematics maps joint angles to the tool pose. The Jacobian linearly relates joint velocities to tool velocity and is the engine of the IK. Inverse kinematics finds joint angles that achieve a desired tool pose. The pipeline generates the sine as a sequence of Cartesian targets, solves IK for each, and streams the resulting joint trajectory to the robot in RViz while tracing the actual drawn path for comparison.

- Forward kinematics: where is the tool, given the joint angles?
- Jacobian: how tool velocity relates to joint velocities
- Inverse kinematics: which joint angles reach a desired tool pose?
- Pipeline: sine path -> IK per point -> joint trajectory -> RViz

Modified (Craig) DH Convention

Four parameters relate each pair of link frames

The Modified DH convention attaches a frame to each link and describes the transform between consecutive frames with four parameters: the link twist α , link length a , link offset d , and joint angle θ . For a revolute joint θ is the variable and the others are fixed. Each link transform is $\text{Rotx}(\alpha_{(i-1)}) \text{Transx}(a_{(i-1)}) \text{Rotz}(\theta_i) \text{Transz}(d_i)$; multiplying the six link transforms gives the full forward kinematics. This is exactly the matrix form used in the course reference notes.

- $\alpha_{(i-1)}$: link twist — angle between successive joint axes
- $a_{(i-1)}$: link length — offset between successive joint axes
- d_i : link offset along the joint axis
- θ_i : joint angle (the variable for a revolute joint)
- Link transform: $\text{Rotx}(\alpha) \cdot \text{Transx}(a) \cdot \text{Rotz}(\theta) \cdot \text{Transz}(d)$

Identified DH Table (verified to $1e-15$ m)

Fitted from the URDF, then checked against it

Because the URDF's link frames are not placed on DH axes, the DH parameters were identified: the four parameters per joint were fitted so that the DH forward kinematics reproduces the URDF forward kinematics, and the residual is at machine precision (about $1e-15$ m over thousands of random configurations). Where parallel axes leave a gauge freedom in the offsets, the physically clean 'concentrated' convention was chosen so the table reads naturally.

- URDF link frames are not on DH axes -> table was identified numerically
- Fit (α , a , d , θ -offset) so DH-FK reproduces URDF-FK
- Residual is machine-zero: $\max |\text{DH-FK} - \text{URDF-FK}| \sim 1e-15$ m
- Physically clean 'concentrated' convention chosen for readability

i	alpha (deg)	a (m)	d (m)	theta off (deg)
1	0	0	0.13056	+90
2	+90	0	0	-90
3	0	-0.1104	0	0
4	0	-0.096	0.06062	-90
5	+90	0	0.07318	+90
6	-90	0	0.0456	0

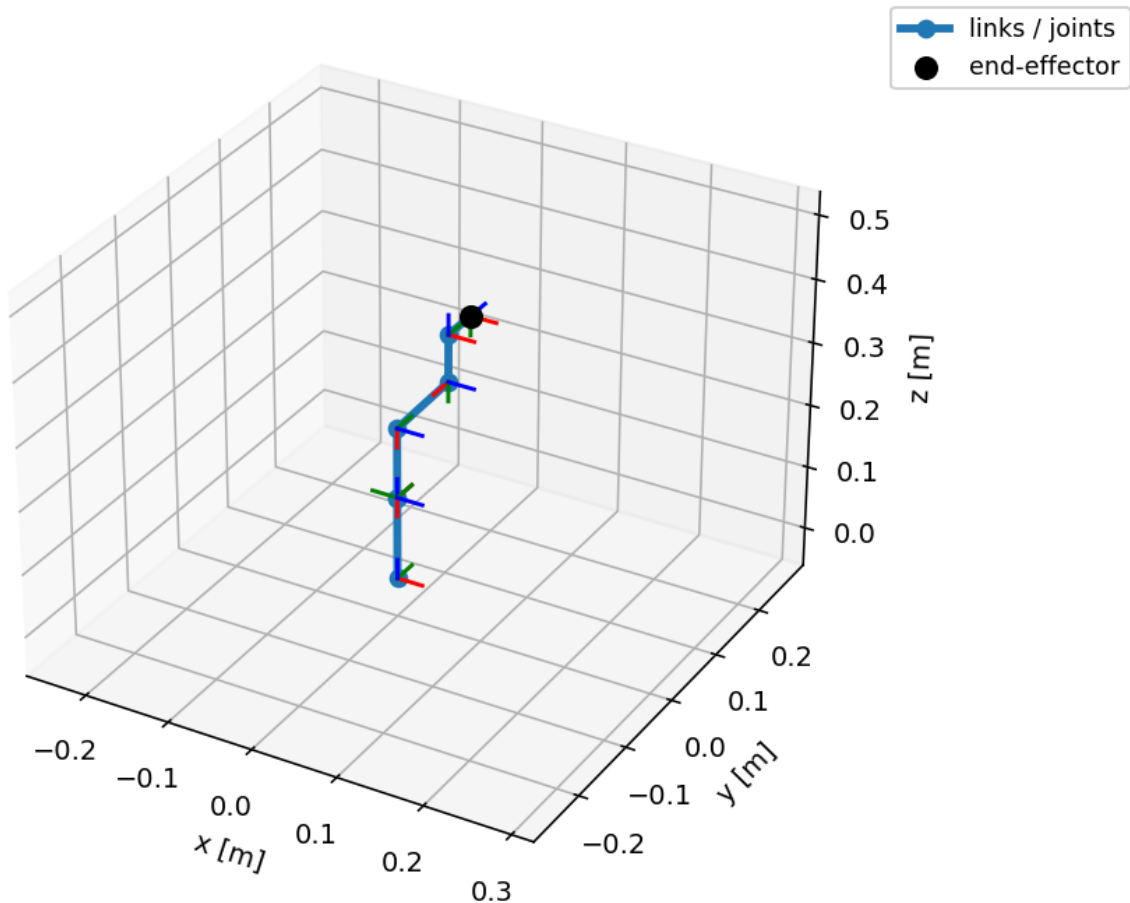
Forward Kinematics

Product of the six link transforms

Multiplying the six modified-DH link transforms yields the base-to-tool transform. At the home configuration (all joint angles zero) the tool sits at (0.061, 0.046, 0.410) m. The figure shows the arm at home with a small coordinate triad drawn at every link frame; ikpy's forward kinematics confirms the same tool position to about $1e-16$ m.

- $T_{EE} = T_1 \cdot T_2 \cdot T_3 \cdot T_4 \cdot T_5 \cdot T_6$
- Home pose (all joints 0): tool at (0.061, 0.046, 0.410) m
- Independently confirmed by ikpy's FK ($\sim 1e-16$ m)

myCobot 280 — home configuration ($q = 0$)
EE at (0.061, 0.046, 0.410) m



Jacobian — Velocity Propagation

Built frame-by-frame, verified against finite differences

The Jacobian is built with the velocity-propagation method: angular and linear velocities are carried outward from the base to the tool, frame by frame. For an all-revolute arm this collapses to the geometric Jacobian whose i -th column is $[z_i \times (p_e - p_i); z_i]$, where z_i and p_i are joint i 's axis and origin and p_e is the tool position. It maps joint rates to the tool twist and is verified against a finite-difference Jacobian to about $2.5e-7$.

- Propagate angular & linear velocity outward: base $\rightarrow$ tool
- For revolute joints reduces to columns $[z_i \times (p_e - p_i); z_i]$
- Maps joint rates to tool twist: $[v; \omega] = J(q) \dot{q}$
- Verified vs numerical Jacobian to $\sim 2.5e-7$

Reachable Workspace (verified)

Where the tool can go — the sine sits well inside it

The reachable workspace is the set of tool positions attainable with some joint configuration inside the URDF limits. Sampling forward kinematics over 40,000 valid configurations gives the cloud shown from three views. The maximum 3D reach is 0.427 m and the maximum horizontal reach 0.301 m, both within the geometric bound. The cloud is verified against ikpy's independent FK to about $1.7e-16$ m, and the demo sine (red) clearly lies inside the reachable region.

- Sampled by FK over 40,000 valid joint configurations
- Max 3D reach 0.427 m; max horizontal reach 0.301 m
- Cross-checked against ikpy FK to $\sim 1.7e-16$ m
- The demo sine lies safely inside the reachable set

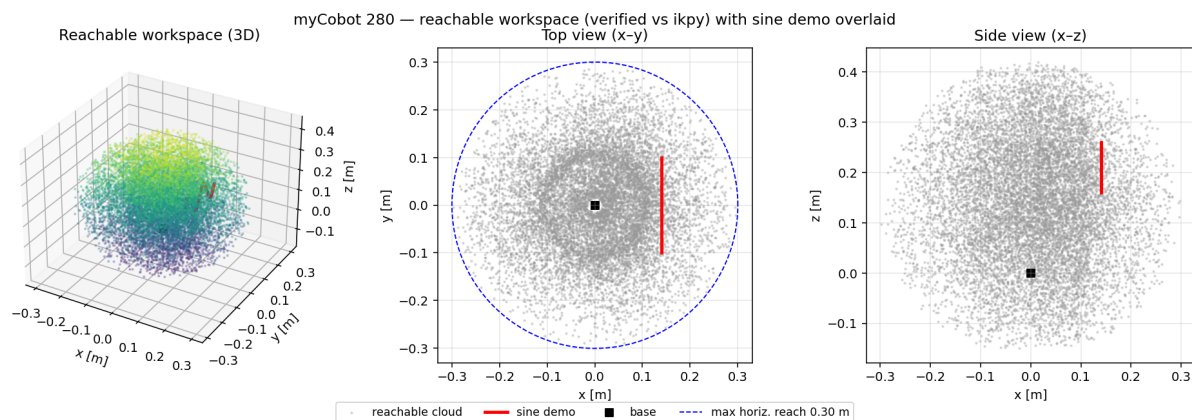


Figure: fig_reachable_workspace.png

Inverse Kinematics — Damped Least Squares

Jacobian-based, robust near singularities

Inverse kinematics is solved iteratively with damped least squares (Levenberg-Marquardt): each step adds $J^T (J J^T + \lambda^2 I)^{-1} e$ to the joint vector, where e stacks the position and orientation error between the current and target tool poses. The damping term λ keeps the update well-conditioned near singularities at the cost of a tiny steady-state error. The solver converges in roughly 16 iterations per waypoint and clamps every solution to the URDF joint limits.

- $q_{k+1} = q_k + J^T (J J^T + \lambda^2 I)^{-1} e$
- $e = [\text{position error} ; \text{orientation error}]$ (6-vector)
- Damping λ keeps updates stable near singularities
- ~16 iterations per waypoint; solutions clamped to joint limits

Configurable Drawing Plane

Same code, any surface

The drawing surface is fully parameterized by an origin and two in-plane axes: the advance direction the pen travels along and the wave direction the sine oscillates along. The target orientation keeps the pen perpendicular to the surface. Because a small arm can only face a surface one way, the solver tries both perpendicular pen directions and keeps whichever is reachable. Switching from a vertical board to a flat plate is only a change of parameters, not code.

- Plane defined by origin + two in-plane axes (advance u , wave v)
- Pen kept perpendicular to the surface (flange z along the normal)
- Solver auto-picks the reachable pen direction (+/- normal)
- Vertical board or flat plate — only the parameters change

Result — Drawing the Sine

Tool traces the target to sub-millimetre accuracy

The demo draws a 20 cm wide, 10 cm tall, two-cycle sine on a vertical board at $x = 0.14$ m. The montage shows the arm progressing through the trajectory as the drawn line grows; the tool holds the pen perpendicular to the board the whole time and the drawn path overlays the target curve to sub-millimetre accuracy.

- 20 cm wide, 10 cm tall, 2-cycle sine on a vertical board
- Pen held perpendicular to the board throughout
- Drawn path overlays the target essentially perfectly

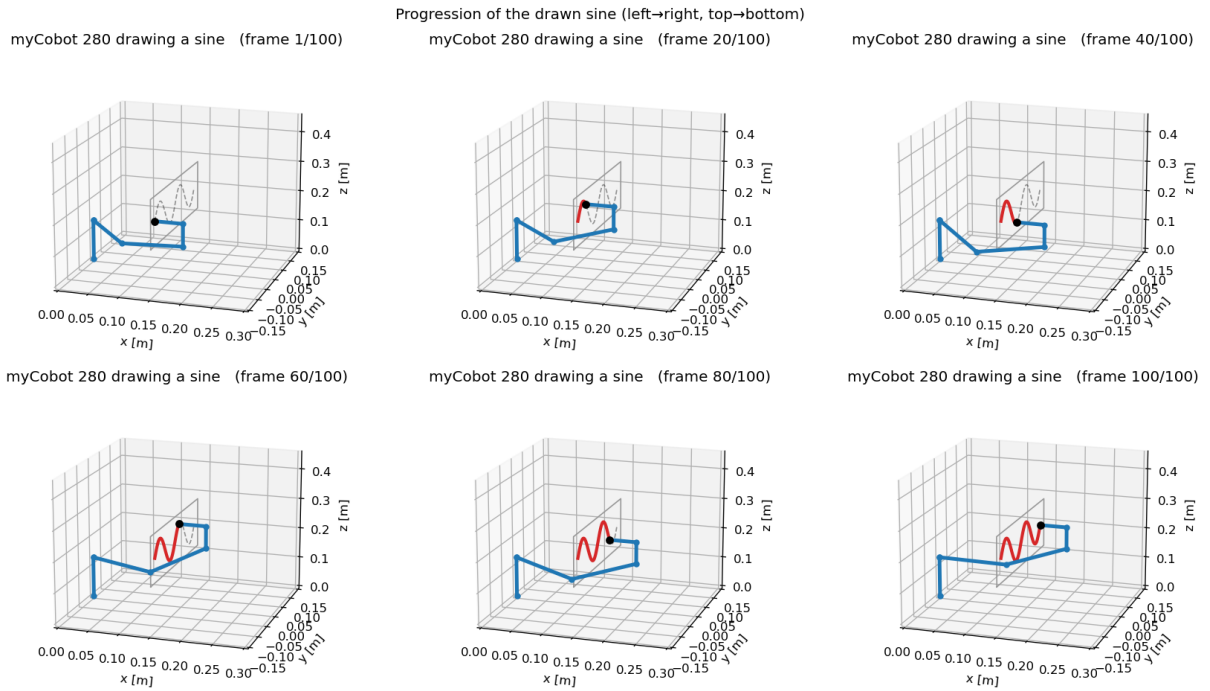


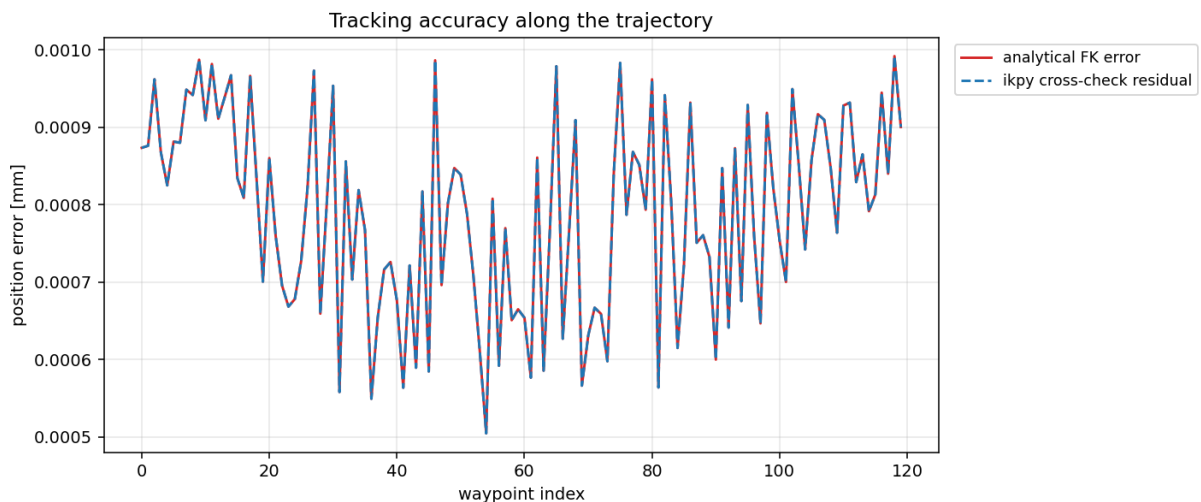
Figure: fig_draw_montage.png

Accuracy and Verification

Numbers behind the result

Each stage is quantified. The DH forward kinematics matches the URDF to about $1e-15$ m; the velocity-propagation Jacobian matches finite differences to about $2.5e-7$; the analytical IK tracks the sine to about 0.001 mm; and an independent ikpy cross-check on every waypoint agrees to about 0.001 mm. The plotted error stays far below one millimetre across the whole trajectory.

- DH-FK vs URDF-FK: $\sim 1e-15$ m
- Velocity-propagation Jacobian vs finite differences: $\sim 2.5e-7$
- Analytical IK tracking error: ~ 0.001 mm
- ikpy cross-check on every waypoint: ~ 0.001 mm



Analytical vs ikpy

Independent methods, same answer

To defend the results, the analytical method is compared against ikpy, an independent kinematics library. The bar chart (log scale) shows every agreement check sitting far below one millimetre, the overlay shows both solvers tracing the same sine, and the per-waypoint error confirms both are sub-millimetre. The two entirely independent implementations agree, which is strong evidence the kinematics are correct.

- My analytical FK/Jacobian/IK vs the ikpy library
- All agreement checks are far below 1 mm (log scale)
- Both solvers trace the same sine

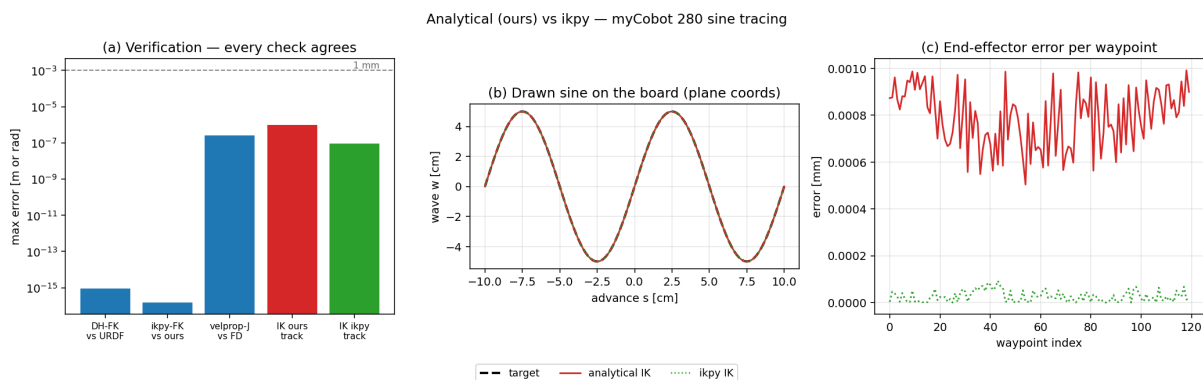


Figure: fig_method_comparison.png

ROS 2 Architecture

Clean, separate Python nodes

The system is organized as clean, single-responsibility ROS 2 (Jazzy) nodes in Python. The math lives in ROS-free modules (kinematics, ik, sine_path) so it can be tested and reused. The sine_ik_node precomputes the IK trajectory and streams joint states at 30 Hz; the path_tracer_node listens to TF and accumulates the real drawn line. A single launch file starts robot_state_publisher, both nodes, and RViz, with the sine fully parameterized.

- kinematics.py / ik.py / sine_path.py — pure math, no ROS
- sine_ik_node — precomputes IK, streams /joint_states at 30 Hz
- path_tracer_node — accumulates the real drawn line from TF
- One launch file -> robot_state_publisher + nodes + RViz

Conclusion and Future Work

The project delivers a fully verified, first-principles kinematics pipeline that draws a sine wave with a myCobot 280 in ROS 2, with an independent cross-check at every stage. The plane and path are general, so other surfaces and shapes follow with minimal change. Natural next steps are running on the physical robot with a real pen, adding Gazebo physics, and shaping the trajectory timing to respect joint velocity limits.

- First-principles kinematics, fully verified, drawing a sine in ROS 2
- General plane + shape: extensible beyond a sine on a board
- Next: run on the physical myCobot 280; add a real pen and contact
- Next: Gazebo physics; trajectory timing / velocity limits

myCobot 280 drawing a sine on a vertical board

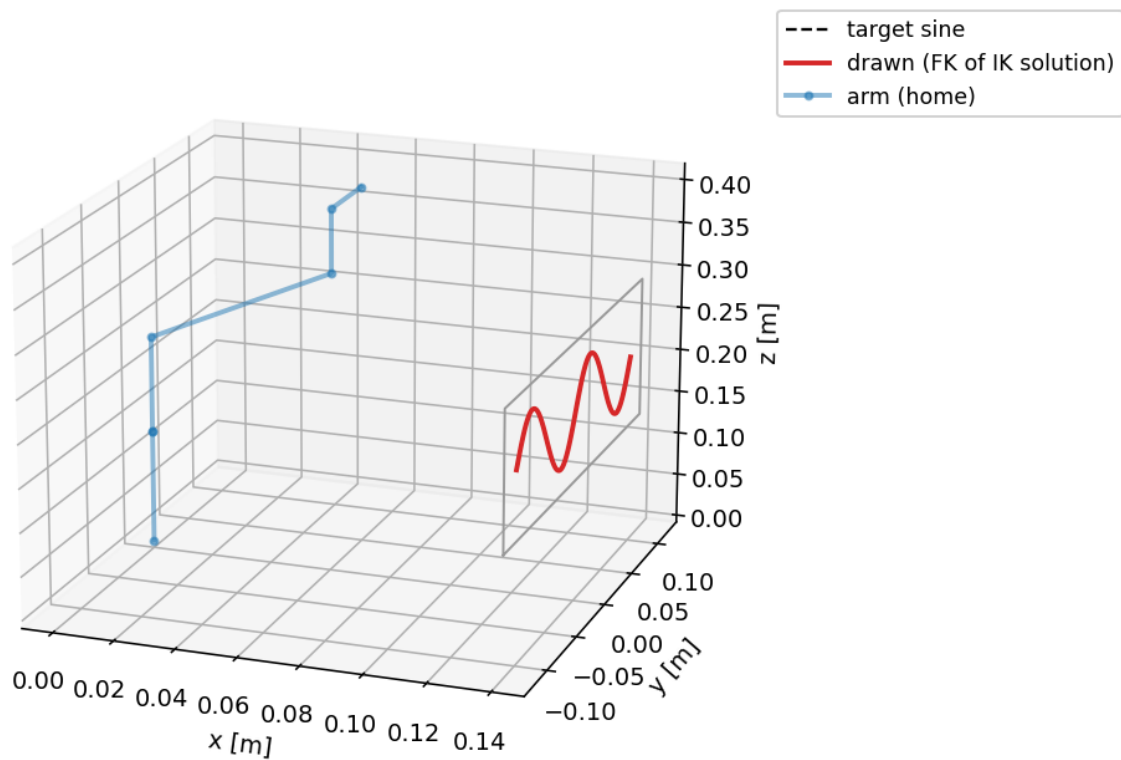


Figure: fig_sine_3d.png